

DBMS & SQL LAB (BCS-552)

B.Tech CSE 5th SEMESTER

SUBMITTED TO:

MR. RAUSHAN RAJ

SUBMITTED BY:

Shivang
(BCS2024352)
SECTION: D

Assignment 1: Creating a Table and Adding Rows

AIM: Create a table, add three rows with one INSERT, and show the table.

TASKS:

TASK 1: Create a table named student with three columns: roll_no (whole number), name (text up to 30 characters) and marks (whole number).

IMPLEMENT:

```
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
```

OUTPUT:

```
Query OK, 0 rows affected
```

TASK 2: Add these three rows to the student table with one INSERT statement: 101, Asha, 78; 102, Rohan, 64; 103, Neha, 88.

IMPLEMENT:

```
INSERT INTO student (roll_no, name, marks) VALUES (101, 'Asha', 78), (102,
'Rohan', 64), (103, 'Neha', 88);
```

OUTPUT:

```
Query OK, 3 rows affected
```

TASK 3: Show all the columns and all the rows of the student table.

IMPLEMENT:

```
SELECT * FROM student;
```

OUTPUT:

```
roll_no name  marks
------- ----- -----
101     Asha  78
102     Rohan 64
103     Neha  88
```

Assignment 2: Updating and Deleting Rows

AIM: Change a value in a row, then delete a row, and check the table after each step.

SET-UP:

```
CREATE TABLE stock (item_code INT, item_name VARCHAR(30), quantity INT);
INSERT INTO stock (item_code, item_name, quantity) VALUES (1, 'Notebook', 50),
(2, 'Pen', 120), (3, 'Bag', 0);
```

TASKS:

TASK 1: Change the quantity of item_code 1 to 80. Then show the table.

IMPLEMENT:

```
UPDATE stock SET quantity = 80 WHERE item_code = 1;
SELECT * FROM stock;
```

OUTPUT:

```
Query OK, 1 row affected

item_code item_name quantity
--------- --------- --------
1         Notebook  80
2         Pen       120
3         Bag       0
```

TASK 2: Add 10 to the quantity of item_code 2. Then show the table.

IMPLEMENT:

```
UPDATE stock SET quantity = quantity + 10 WHERE item_code = 2;
SELECT * FROM stock;
```

OUTPUT:

```
Query OK, 1 row affected

item_code item_name quantity
--------- --------- --------
1         Notebook  80
2         Pen       130
3         Bag       0
```

TASK 3: Delete the row of item_code 3 from the stock table. Then show the table.

IMPLEMENT:

```
DELETE FROM stock WHERE item_code = 3;
SELECT * FROM stock;
```

OUTPUT:

```
Query OK, 1 row affected

item_code item_name quantity
--------- --------- --------
1         Notebook  80
2         Pen       130
```

Assignment 3: Selecting Columns and Rows with WHERE

AIM: Show some columns, and show only the rows that match a condition.

SET-UP:

```
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
INSERT INTO student (roll_no, name, marks) VALUES (101, 'Asha', 78), (102,
'Rohan', 64), (103, 'Neha', 88), (104, 'Deepak', 52);
```

TASKS:

TASK 1: Show only the name and marks of all the students.

IMPLEMENT:

```
SELECT name, marks FROM student;
```

OUTPUT:

```
name   marks
------ -----
Asha   78
Rohan  64
Neha   88
Deepak 52
```

TASK 2: Show all the columns of the students whose marks are greater than 60.

IMPLEMENT:

```
SELECT * FROM student WHERE marks > 60;
```

OUTPUT:

```
roll_no name  marks
------- ----- -----
101     Asha  78
102     Rohan 64
103     Neha  88
```

TASK 3: Show the name of the student whose roll_no is 103.

IMPLEMENT:

```
SELECT name FROM student WHERE roll_no = 103;
```

OUTPUT:

```
name
----
Neha
```

Assignment 4: Combining Conditions

AIM: Combine conditions with AND, check a range with BETWEEN, and check a list with IN.

SET-UP:

```
CREATE TABLE player (name VARCHAR(30), team VARCHAR(20), runs INT);
INSERT INTO player (name, team, runs) VALUES ('Asha', 'Eagles', 45), ('Meera',
'Tigers', 72), ('Arjun', 'Eagles', 30), ('Kiran', 'Lions', 88);
```

TASKS:

TASK 1: Show the name and runs of the players from Eagles who scored more than 35 runs. Use AND.

IMPLEMENT:

```
SELECT name, runs FROM player WHERE team = 'Eagles' AND runs > 35;
```

OUTPUT:

```
name runs
---- ----
Asha 45
```

TASK 2: Show the name and runs of the players whose runs are between 40 and 80. Use BETWEEN.

IMPLEMENT:

```
SELECT name, runs FROM player WHERE runs BETWEEN 40 AND 80;
```

OUTPUT:

```
name  runs
----- ----
Asha  45
Meera 72
```

TASK 3: Show the name and team of the players who play for Tigers or Lions. Use IN.

IMPLEMENT:

```
SELECT name, team FROM player WHERE team IN ('Tigers', 'Lions');
```

OUTPUT:

```
name  team
----- ------
Meera Tigers
Kiran Lions
```

Assignment 5: Pattern Matching with LIKE

AIM: Find titles that match a pattern with LIKE.

SET-UP:

```
CREATE TABLE book (title VARCHAR(40), category VARCHAR(20), copies INT);
INSERT INTO book (title, category, copies) VALUES ('Database Systems',
'Computer', 3), ('Digital Electronics', 'Electronics', 2), ('Data Structures
in C', 'Computer', 2), ('Short Stories', 'Literature', 1);
```

TASKS:

TASK 1: Show the titles that start with Data. Use LIKE with the pattern 'Data%'.

IMPLEMENT:

```
SELECT title FROM book WHERE title LIKE 'Data%';
```

OUTPUT:

```
title
--------------------
Database Systems
Data Structures in C
```

TASK 2: Show the titles that end with the letter C. Use LIKE with the pattern '%C'.

IMPLEMENT:

```
SELECT title FROM book WHERE title LIKE '%C';
```

OUTPUT:

```
title
--------------------
Data Structures in C
```

TASK 3: Show the titles whose second letter is h. Use LIKE with the pattern '_h%'.

IMPLEMENT:

```
SELECT title FROM book WHERE title LIKE '_h%';
```

OUTPUT:

```
title
-------------
Short Stories
```

Assignment 6: Sorting Rows and Removing Repeats

AIM: Sort the rows, remove repeated values, and show only some rows.

SET-UP:

```
CREATE TABLE product (product_name VARCHAR(30), category VARCHAR(20), price
INT);
INSERT INTO product (product_name, category, price) VALUES ('Pen',
'Stationery', 15), ('Notebook', 'Stationery', 60), ('Calculator',
'Electronics', 850), ('Bag', 'Office', 500);
```

TASKS:

TASK 1: Show the product name and price of all products, sorted by price from highest to lowest.

IMPLEMENT:

```
SELECT product_name, price FROM product ORDER BY price DESC;
```

OUTPUT:

```
product_name price
------------ -----
Calculator   850
Bag          500
Notebook     60
Pen          15
```

TASK 2: Show each category once, sorted by category name.

IMPLEMENT:

```
SELECT DISTINCT category FROM product ORDER BY category;
```

OUTPUT:

```
category
-----------
Electronics
Office
Stationery
```

TASK 3: Show the product name and price of the two most expensive products, sorted by price from highest to lowest.

IMPLEMENT:

```
SELECT product_name, price FROM product ORDER BY price DESC LIMIT 2;
```

OUTPUT:

```
product_name price
------------ -----
Calculator   850
Bag          500
```

Assignment 7: Using Aggregate Functions

AIM: Use one aggregate function in each task.

SET-UP:

```
CREATE TABLE sales (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO sales (salesperson, region, amount) VALUES ('Deepak', 'North',
12000), ('Pooja', 'South', 8500), ('Deepak', 'South', 9500), ('Lata', 'North',
11000);
```

TASKS:

TASK 1: Show how many sales rows there are. Name the column num_sales.

IMPLEMENT:

```
SELECT COUNT(*) AS num_sales FROM sales;
```

OUTPUT:

```
num_sales
---------
4
```

TASK 2: Show the total of all the amounts. Name the column total.

IMPLEMENT:

```
SELECT SUM(amount) AS total FROM sales;
```

OUTPUT:

```
total
-----
41000
```

TASK 3: Show the highest amount. Name the column highest.

IMPLEMENT:

```
SELECT MAX(amount) AS highest FROM sales;
```

OUTPUT:

```
highest
-------
12000
```

Assignment 8: Grouping Rows and Using HAVING

AIM: Group the rows by region and check the totals of each group.

SET-UP:

```
CREATE TABLE shop_sale (salesperson VARCHAR(20), region VARCHAR(20), amount
INT);
INSERT INTO shop_sale (salesperson, region, amount) VALUES ('Deepak', 'North',
12000), ('Pooja', 'South', 8500), ('Deepak', 'South', 9500), ('Lata', 'North',
11000);
```

TASKS:

TASK 1: Show each region with the number of sales in it. Use GROUP BY region and sort by region.

IMPLEMENT:

```
SELECT region, COUNT(*) AS num_sales FROM shop_sale GROUP BY region ORDER BY
region;
```

OUTPUT:

```
region num_sales
------ ---------
North  2
South  2
```

TASK 2: Show each region with the total amount in it. Use GROUP BY region and sort by region.

IMPLEMENT:

```
SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region ORDER BY
region;
```

OUTPUT:

```
region total
------ -----
North  23000
South  18000
```

TASK 3: Show only the regions whose total amount is more than 20000. Use HAVING and sort by region.

IMPLEMENT:

```
SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region HAVING
SUM(amount) > 20000 ORDER BY region;
```

OUTPUT:

```
region total
------ -----
North  23000
```

Assignment 9: Aliases and CASE

AIM: Give columns new names and show a value chosen by a condition.

SET-UP:

```
CREATE TABLE exam_result (name VARCHAR(20), marks INT);
INSERT INTO exam_result (name, marks) VALUES ('Asha', 92), ('Rohan', 47),
('Neha', 68), ('Meera', 33);
```

TASKS:

TASK 1: Show the name and marks. Name the columns student_name and score, using AS.

IMPLEMENT:

```
SELECT name AS student_name, marks AS score FROM exam_result;
```

OUTPUT:

```
student_name score
------------ -----
Asha         92
Rohan        47
Neha         68
Meera        33
```

TASK 2: Show the name and a column named result. The result is Pass if the marks are 40 or more, otherwise Fail. Use CASE.

IMPLEMENT:

```
SELECT name, CASE WHEN marks >= 40 THEN 'Pass' ELSE 'Fail' END AS result FROM
exam_result;
```

OUTPUT:

```
name  result
----- ------
Asha  Pass
Rohan Pass
Neha  Pass
Meera Fail
```

TASK 3: Show the name, and the marks with 5 bonus marks added. Name this column with_bonus.

IMPLEMENT:

```
SELECT name, marks + 5 AS with_bonus FROM exam_result;
```

OUTPUT:

```
name  with_bonus
----- ----------
Asha  97
Rohan 52
Neha  73
Meera 38
```

Assignment 10: Joining Two Tables with INNER JOIN

AIM: Join a department table and an employee table on dept_id.

SET-UP:

```
CREATE TABLE department (dept_id INT, dept_name VARCHAR(20));
INSERT INTO department (dept_id, dept_name) VALUES (10, 'Accounts'), (20,
'Sales'), (40, 'Research');
CREATE TABLE employee (emp_name VARCHAR(30), dept_id INT, salary INT);
INSERT INTO employee (emp_name, dept_id, salary) VALUES ('Anita', 10, 90000),
('Lata', 20, 55000), ('Deepak', 10, 60000), ('Pooja', 20, 42000);
```

TASKS:

TASK 1: Show each employee name with the name of their department. Use INNER JOIN on dept_id and sort by employee name.

IMPLEMENT:

```
SELECT e.emp_name, d.dept_name
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.emp_name;
```

OUTPUT:

```
emp_name dept_name
-------- ---------
Anita    Accounts
Deepak   Accounts
Lata     Sales
Pooja    Sales
```

TASK 2: Show the names of the employees in the Sales department. Use INNER JOIN and WHERE, and sort by employee name.

IMPLEMENT:

```
SELECT e.emp_name
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
WHERE d.dept_name = 'Sales'
ORDER BY e.emp_name;
```

OUTPUT:

```
emp_name
--------
Lata
Pooja
```

TASK 3: Show the employee name, the department name and the salary, sorted by salary from highest to lowest.

IMPLEMENT:

```
SELECT e.emp_name, d.dept_name, e.salary
FROM employee e
INNER JOIN department d ON e.dept_id = d.dept_id
ORDER BY e.salary DESC;
```

OUTPUT:

```
emp_name dept_name salary
-------- --------- ------
Anita    Accounts  90000
Deepak   Accounts  60000
Lata     Sales     55000
Pooja    Sales     42000
```

Assignment 11: Other Kinds of Joins

AIM: Use a left join, a cross join and a natural join on two small tables.

SET-UP:

```
CREATE TABLE team (team_id INT, team_name VARCHAR(20));
INSERT INTO team (team_id, team_name) VALUES (1, 'Eagles'), (2, 'Tigers'), (3,
'Lions');
CREATE TABLE player (player_name VARCHAR(30), team_id INT, age INT);
INSERT INTO player (player_name, team_id, age) VALUES ('Asha', 1, 24),
('Meera', 1, 22), ('Arjun', 2, 27);
```

TASKS:

TASK 1: Show each team name with the name of its players, including the team with no players. Use LEFT JOIN with team on the left, and sort by team name, then by player name.

IMPLEMENT:

```
SELECT t.team_name, p.player_name
FROM team t
LEFT JOIN player p ON t.team_id = p.team_id
ORDER BY t.team_name, p.player_name;
```

OUTPUT:

```
team_name player_name
--------- -----------
Eagles    Asha
Eagles    Meera
Lions
Tigers    Arjun
```

TASK 2: Show every pair of team name and player name. Use CROSS JOIN between team and player, and sort by team name, then by player name.

IMPLEMENT:

```
SELECT t.team_name, p.player_name
FROM team t
CROSS JOIN player p
ORDER BY t.team_name, p.player_name;
```

OUTPUT:

```
team_name player_name
--------- -----------
Eagles    Arjun
Eagles    Asha
Eagles    Meera
Lions     Arjun
Lions     Asha
Lions     Meera
Tigers    Arjun
Tigers    Asha
Tigers    Meera
```

TASK 3: Show each player name with the team name. Use NATURAL JOIN between player and team, and sort by player name.

IMPLEMENT:

```
SELECT player_name, team_name
FROM player
NATURAL JOIN team
ORDER BY player_name;
```

OUTPUT:

```
player_name team_name
----------- ---------
Arjun       Tigers
Asha        Eagles
Meera       Eagles
```

Assignment 12: Using a Subquery

AIM: Use one query inside another query, with IN.

SET-UP:

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

TASKS:

TASK 1: Show the names in club_a that are also in club_b. Use a subquery after IN.

IMPLEMENT:

```
SELECT student_name FROM club_a
WHERE student_name IN (SELECT student_name FROM club_b);
```

OUTPUT:

```
student_name
------------
Neha
```

TASK 2: Show how many names are in club_b. Name the column members.

IMPLEMENT:

```
SELECT COUNT(*) AS members FROM club_b;
```

OUTPUT:

```
members
-------
3
```

TASK 3: Show the names in club_a, sorted by name.

IMPLEMENT:

```
SELECT student_name FROM club_a ORDER BY student_name;
```

OUTPUT:

```
student_name
------------
Asha
Neha
Rohan
```

Assignment 13: UNION and INTERSECT

AIM: Combine two lists of names, and find the names in both lists.

SET-UP:

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

TASKS:

TASK 1: Show all the names from club_a and club_b with each name once, sorted by name. Use UNION.

IMPLEMENT:

```
SELECT student_name FROM club_a
UNION
SELECT student_name FROM club_b
ORDER BY student_name;
```

OUTPUT:

```
student_name
------------
Asha
Meera
Neha
Pooja
Rohan
```

TASK 2: Show the names that are in both clubs, sorted by name. Use INTERSECT.

IMPLEMENT:

```
SELECT student_name FROM club_a
INTERSECT
SELECT student_name FROM club_b
ORDER BY student_name;
```

OUTPUT:

```
student_name
------------
Neha
```

TASK 3: Show all the names from both clubs, including repeated names, sorted by name. Use UNION ALL.

IMPLEMENT:

```
SELECT student_name FROM club_a
UNION ALL
SELECT student_name FROM club_b
ORDER BY student_name;
```

OUTPUT:

```
student_name
------------
Asha
Meera
Neha
Neha
Pooja
Rohan
```

Assignment 14: Creating and Using a View

AIM: Save a query as a view, then read the view.

SET-UP:

```
CREATE TABLE score (name VARCHAR(30), subject VARCHAR(20), marks INT);
INSERT INTO score (name, subject, marks) VALUES ('Asha', 'Maths', 78),
('Rohan', 'Maths', 64), ('Neha', 'Maths', 88), ('Kiran', 'Maths', 52);
```

TASKS:

TASK 1: Create a view named high_scores that shows the name and marks of the rows with marks 70 or more.

IMPLEMENT:

```
CREATE VIEW high_scores AS
SELECT name, marks FROM score WHERE marks >= 70;
```

OUTPUT:

```
Query OK, 0 rows affected
```

TASK 2: Show all the rows of the view high_scores.

IMPLEMENT:

```
SELECT * FROM high_scores;
```

OUTPUT:

```
name marks
---- -----
Asha 78
Neha 88
```

TASK 3: Show only the names in the view high_scores, sorted by name.

IMPLEMENT:

```
SELECT name FROM high_scores ORDER BY name;
```

OUTPUT:

```
name
----
Asha
Neha
```

Assignment 15: Indexes and Adding a Row

AIM: Create an index on a column, read rows by that column, and add a row.

SET-UP:

```
CREATE TABLE customer (customer_id INT PRIMARY KEY, name VARCHAR(30) NOT NULL,
city VARCHAR(20));
INSERT INTO customer (customer_id, name, city) VALUES (1, 'Kiran', 'Kochi'),
(2, 'Deepak', 'Delhi'), (3, 'Lata', 'Mumbai'), (4, 'Asha', 'Delhi');
```

TASKS:

TASK 1: Create an index named idx_city on the city column of the customer table.

IMPLEMENT:

```
CREATE INDEX idx_city ON customer (city);
```

OUTPUT:

```
Query OK, 0 rows affected
```

TASK 2: Show the name of the customers who live in Delhi, sorted by name.

IMPLEMENT:

```
SELECT name FROM customer WHERE city = 'Delhi' ORDER BY name;
```

OUTPUT:

```
name
------
Asha
Deepak
```

TASK 3: Add a customer with customer_id 5, name Arjun and city Pune with one INSERT. Then show the table.

IMPLEMENT:

```
INSERT INTO customer (customer_id, name, city) VALUES (5, 'Arjun', 'Pune');
SELECT * FROM customer;
```

OUTPUT:

```
Query OK, 1 row affected

customer_id name   city
----------- ------ ------
1           Kiran  Kochi
2           Deepak Delhi
3           Lata   Mumbai
4           Asha   Delhi
5           Arjun  Pune
```